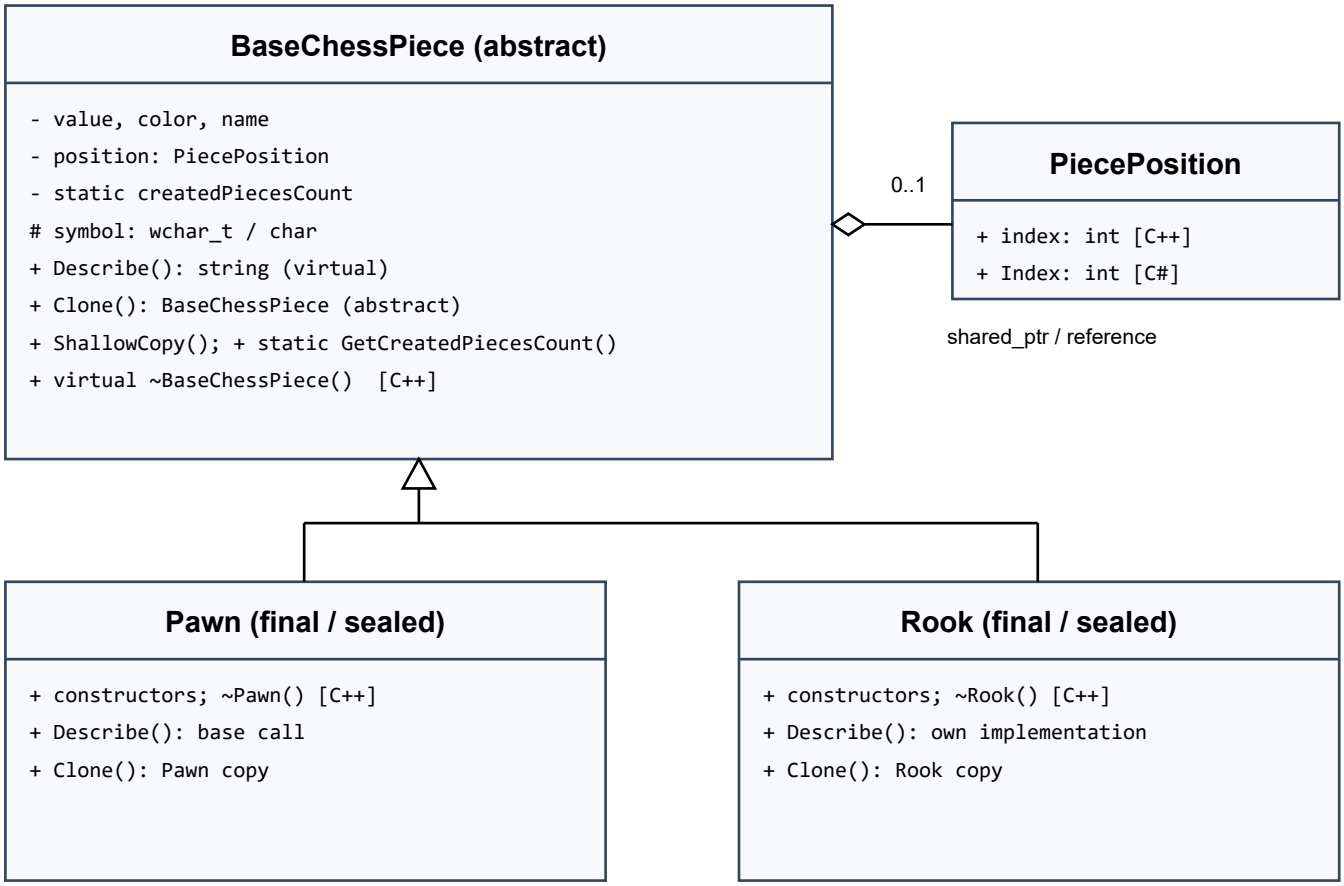
Лабораторная работа 4

Наследование. Предметная область: «Шахматы 2.0». C++ и C#.

Работа продолжает ЛР3: существующая фигура заменена абстрактным BaseChessPiece. Pawn и Rook представляют пешку и ладью. Доска работает с базовым типом, сохраняя размещение фигур и ограничение их суммарной ценности.

1. Иерархия и видимость



- private # protected + public

Обозначения соответствуют C++ и C#: symbol — _wcharChessPiece / _charChessPiece; value, color, name — ценность, цвет и название фигуры. Деструкторы указаны только для C++.

protected: символ фигуры нужен обоим наследникам в Describe(). При private прямой доступ из наследника невозможен; public разрешил бы внешнему коду менять символ. Остальное состояние закрыто, чтение и изменение позиции выполняются через методы или свойства.

PiecePosition содержит только индекс клетки. Для сравнения способов копирования используются фигуры до размещения на доске. Дополнительные правила перемещения не вводятся.

2. Конструкторы и ресурсы

В C++ конструкторы по умолчанию и с параметрами вызывают конструктор BaseChessPiece в списке инициализации. Копирующие и перемещающие конструкторы наследников передают объект базовому конструктору; их сообщения позволяют увидеть порядок вызовов.

```
Pawn::Pawn(bool isWhitePiece) :
    BaseChessPiece(L'P', 1, isWhitePiece, "Pawn")
{
    std::wcout << L"Pawn was created (parameters)" << std::endl;
}
Pawn copied(original);
Pawn moved(std::move(copied));
```

Начало фактического вывода C++:

```
BaseChessPiece was created
Pawn was created (default)
BaseChessPiece was copied
Pawn was copied
BaseChessPiece was moved
Pawn was moved
```

Сначала выполняется конструктор базового класса, затем производного. При уничтожении порядок обратный. В массиве std::unique_ptr<BaseChessPiece> хранятся Pawn и Rook; при выходе из блока элементы освобождаются через базовый тип.

```
Rook was destroyed
BaseChessPiece was destroyed
Pawn was destroyed
BaseChessPiece was destroyed
```

Виртуальный деструктор: ~BaseChessPiece() объявлен virtual, деструкторы Pawn и Rook — override. Удаление наследника через указатель на базовый класс без виртуального деструктора вызвало бы неопределённое поведение.

Правило пяти: BaseChessPiece определяет копирующий конструктор и присваивание с независимой позицией, перемещающий конструктор и присваивание с переносом ресурсов, а также виртуальный деструктор. Наследники выводят сообщения в конструкторах и деструкторах, а оба оператора присваивания оставлены = default: они используют операции базы. PiecePosition следует правилу нуля.

this: операторы проверяют this == &other и возвращают *this; (assigned = original) = assigned показывает цепочку и самоприсваивание. Clone() передаёт *this конструктору копии. После перемещения исходная позиция читается как -1; SetIndexInChessboard() при необходимости создаёт её заново.

3. Поверхностная и глубокая копия

В C++ позиция хранится в std::shared_ptr<PiecePosition>. ShallowCopy() получает объект нужного типа через Clone(), затем заменяет его независимую позицию общим указателем. Поля фигуры копируются, позиция становится общей.

```
std::unique_ptr<BaseChessPiece> BaseChessPiece::ShallowCopy() const
{
    auto copy = Clone();
    copy->_position = _position;
    return copy;
}
```

При глубоком копировании базовый конструктор создаёт собственный PiecePosition. Копирующее присваивание делает то же. Каждый Clone() создаёт копию своего производного типа, сохраняя полиморфизм.

```
// Инициализатор копирующего конструктора BaseChessPiece:
_position{std::make_shared<PiecePosition>(
    PiecePosition{other.GetIndexInChessboard()})}

std::unique_ptr<BaseChessPiece> Pawn::Clone() const
{
    return std::make_unique<Pawn>(*this);
}
```

В C# ShallowCopy() вызывает MemberwiseClone(): поле _position остаётся общей ссылкой. В конструкторе копирования выделяется отдельный объект:

```
_position = new PiecePosition { Index = other.IndexInChessboard };

public override BaseChessPiece Clone() => new Pawn(this);
```

Фактический результат одинаков в обоих проектах:

```
Shallow: original = 16, copy = 16
Deep: original = 16, copy = 24
```

Исходный индекс — 8. Поверхностной копии присваивается 16: оригинал тоже получает 16. Затем глубокая копия получает 24, а у оригинала остаётся 16. Копии не размещаются на доске: общий индекс служит только демонстрации.

Управление памятью в C++ выполняют make_unique, make_shared и умные указатели. В исходном коде C++ нет ручных new и delete. Методы чтения помечены const, неизменяемые входные объекты передаются по const&.

4. Перенос на C# и проверка

В C# базовый класс объявлен abstract, наследники — sealed. Конструкторы используют : base(...), методы — override. Pawn.Describe() вызывает base.Describe(); Rook.Describe() формирует собственное описание. Память управляется сборщиком мусора: операции перемещения и виртуальный деструктор из C++ не переносятся.

Equals() сравнивает тип, символ, ценность, цвет, имя и индекс. GetHashCode() использует те же признаки. Поэтому независимые копии равны по содержимому, а изменение позиции нарушает равенство.

```
Equal copies = True, same hash = True
After change = False, different type = False
```

Позиция изменяема, поэтому хеш может меняться. Фигуру не следует изменять, пока она используется как ключ хеш-коллекции.

Объявление using IssueLog log освобождает существующий ресурс журнала при выходе из области видимости. Последние строки фактического вывода:

```
Created pieces = 9
 Program end
[-] журнал закрыт: LOG
```

Статические члены: _createdPiecesCount и GetCreatedPiecesCount() считают все созданные фигуры, включая копии. В C# MemberwiseClone() не вызывает конструктор, поэтому счётчик увеличивается в ShallowCopy(). В конце демонстраций: C++ — 11, C# — 9.

Строки: в C++ применены std::string, конкатенация имени и цвета, find(' '), substr(0, separator) и сравнение с «Pawn». В C# показаны аналогичные операции со string, IndexOf и Substring.

Сборка и результаты проверки

Проверено на Windows: g++ (C++20) с -Wall -Wextra -Werror -pedantic и .NET SDK 10.0.401. Обе сборки завершились без предупреждений и ошибок. Временные проверки подтвердили независимость глубоких копий и присваивания, совместное состояние поверхностных копий, перемещение и самоприсваивание C++, полиморфизм обоих типов, равенство и согласованность хешей C#, размещение на индексах 0 и 63.

Точные команды приведены в README.md каждой ветки. C++:

```
cd console/scripts
g++ -std=c++20 main.cpp GameManagement.cpp Chessboard.cpp `
    BaseChessPiece.cpp Pawn.cpp Rook.cpp -o app.exe
.\app.exe
```

Длинные команды используют перенос строк PowerShell; в README они записаны одной строкой.

C#:

```
dotnet build "lab work on c#/ChessApp.csproj" -c Release
dotnet run --project "lab work on c#/ChessApp.csproj" `
    -c Release --no-build --no-restore
```